

1. Installa Arduino IDE (se non ce l'hai già)

Vai su <https://www.arduino.cc/en/software>, scarica la versione per il tuo sistema operativo.

2. Verifica che il PC veda l'Arduino

Apri Arduino IDE. In alto, c'è un menù **Strumenti** → **Porta** (Tools → Port).

Quello che dovresti vedere dipende dal sistema:

- **Windows:** una voce tipo COM3 con scritto accanto "Arduino Uno".
- **Mac:** una voce tipo `/dev/cu.usbmodem1411` o simile.

Selezionala. Poi in **Strumenti** → **Scheda** (Tools → Board) seleziona **"Arduino Uno"**

3. Apri lo sketch attuale

In Arduino IDE: **File** → **Apri**, e naviga fino al file `prova1_mvd.ino` che hai nel repo. Si apre l'editor con il codice attuale

NUOVO

Aggiungiamo all'Arduino la capacità di **ascoltare** sulla seriale. Adesso lui solo *parla* (stampa `y = . . .`).

Vogliamo che ascolti due caratteri (Serial Monitor):

- 1, 2 o 3 → fa partire la rampa di una delle tre cadute
- R → resetta tutto

In requirements aggiunto
pyserial==3.5

pyserial è la libreria Python che permette di parlare con la seriale USB.

Dal terminale lancia:
pip install pyserial==3.5

NUOVO `sensor_monitor.py`
«backend/sensor_monitor.py - V2.2 con lettura seriale
Arduino Monitor per ascolto CONTINUO del sensore
collegato al backend via USB.
Sostituisce `read_sensor()` simulato con lettura reale da
Arduino. Arduino gira lo sketch prova1_mvd: comando '2'
avvia la rampa, 'R' resetta».

Creazione virtual environment (1 volta):

```
cd C:\Users\...\backend  
python -m venv venv
```

Attivazione:
venv\Scripts\activate

Si vedrà:
(venv) C:\Users\...\Cai-CSMT\backend>

Installazione requisiti (1 volta):
pip install -r requirements.txt

Per prova lanciare:
python test_serial.py

```
...  
===== RISULTATO =====  
Lecture totali: 84  
Picco: 800.0  
Offset picco: 4142ms  
Durata: 7955ms
```

Far partire il server React

React/Vite girano su **Node.js**, che è un motore JavaScript per girare codice fuori dal browser.

Se devi installarlo

Vai su <https://nodejs.org>. Vedrai due opzioni di download: **LTS** e **Current**. Prendi **LTS** (è la versione "stabile", quella consigliata). Scarica l'installer Windows, lancialo, click click click "Next" su tutto. Si installa anche **npm** (Node Package Manager), che è quello che ti serve davvero.

Quando finisce, **chiudi e riapri Anaconda Prompt** (è importante: i programmi appena installati non sono visibili nei terminali già aperti). Poi verifica:

```
Anaconda Prompt
(base) C:\Users\aless>node --version
v22.17.0

(base) C:\Users\aless>npm --version
10.9.2
```

Setup iniziale del frontend (una volta sola)

Stesso ragionamento di `pip install -r requirements.txt` per Python: anche React ha le sue librerie da scaricare. Le decide il file `package.json` nel frontend, e si installano con `npm install`.

In Anaconda Prompt, **apri un nuovo terminale** (lascia stare quello dove hai il backend, ne servono due in parallelo):

```
cd C:\percorso\fino\a\Cai-CSMT\frontend
npm install
```

Far partire il sito

Sempre in quel terminale, dentro `frontend/`:

```
npm run dev
```

```
C:\WINDOWS\system32\cmd.
VITE v5.4.21 ready in 369 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Quel `http://localhost:5173/` è l'indirizzo del tuo sito. **Apri il browser** (Chrome, Firefox, qualunque) **e vai su quell'URL**. Dovresti vedere la pagina React con il form.

TO DO: definire bene i campi essenziali ed informativi per il CAI riguardo le prove

MVD2555 Configurator
V2.2 - Con monitoraggio caduta da sensore

Carica Preset
-- Seleziona un preset --

Corda
es. 10mm

Assicuratore
es. CAMP

Operatore
Nome operatore

Salva come Preset
Nome preset (opzionale)

Lascia vuoto se non vuoi salvare come preset

☒ **CONFERMA**

CADUTA EFFETTUATA

Compila il form e premi CONFERMA per avviare il monitoraggio del sensore

Esporta dati
Scarica i dati delle cadute in formato CSV (apribile direttamente in Excel).

Scarica riepilogo (tutte le cadute)

Scarica ultima caduta (serie temporale)

NUOVO: salva cadute come csv su raspberry oltre che json.
Tasto download dal sito per file csv con tutte le cadute fatte e salvate

20°C
Soleggiato

Cerca

ITA

17:54
02/05/2026

PER IL FUTURO, PER FAR FUNZIONARE RASPBERRY AL POSTO DEL PC

NUOVO: file **cai-csmt.service** quando lo metteremo sull'raspberry perché non potremo lanciare il file python `main.py` da là

Cosa fa, sezione per sezione

[Unit] — aspetta che la rete sia pronta prima di partire

[Service] — il cuore. `Type=simple` dice che il programma gira in primo piano (non si "demonizza"). `User=pi` è l'utente che esegue il programma (di default sui Raspberry l'utente è `pi`; se ne avete uno diverso lo cambiate qui). `WorkingDirectory` è la cartella da cui parte (importante perché `main.py` usa percorsi relativi tipo `./data/`). `ExecStart` è il comando vero. `Restart=on-failure` significa "se crasha, riavvialo automaticamente dopo 5 secondi

[Install] — `WantedBy=multi-user.target` significa "fai partire questo servizio quando il Raspberry arriva al runlevel multi-utente normale", ovvero al boot completato.

Come si attiva (sul Raspberry, in futuro)

Sequenza di comandi da fare una sola volta:

```
bash

# Copia il file nel posto giusto
sudo cp cai-csmt.service /etc/systemd/system/

# Dice a systemd di rileggere i servizi
sudo systemctl daemon-reload

# Abilita il servizio (parte ad ogni boot)
sudo systemctl enable cai-csmt.service

# Lo avvia subito senza dover riavviare
sudo systemctl start cai-csmt.service
```

Da quel momento, ogni volta che il Raspberry si accende, il backend parte automaticamente. Fine. L'operatore non vede terminali, comandi, niente.

E il sito?

Sul Raspberry **non** lancerete `npm run dev`. Si fa così, una volta sola in fase di setup:

1. Sul PC di sviluppo: `cd frontend; npm run build`. Questo genera una cartella `frontend/dist/` con il sito "compilato" (HTML+CSS+JS pronti).
2. Si copia il contenuto di `frontend/dist/` dentro `backend/static/` del Raspberry.
3. Nel `main.py` c'è già la riga `app.mount("/", StaticFiles(directory="static", html=True))`. Questa fa sì che FastAPI serva quei file statici sulla porta 8000 stessa.

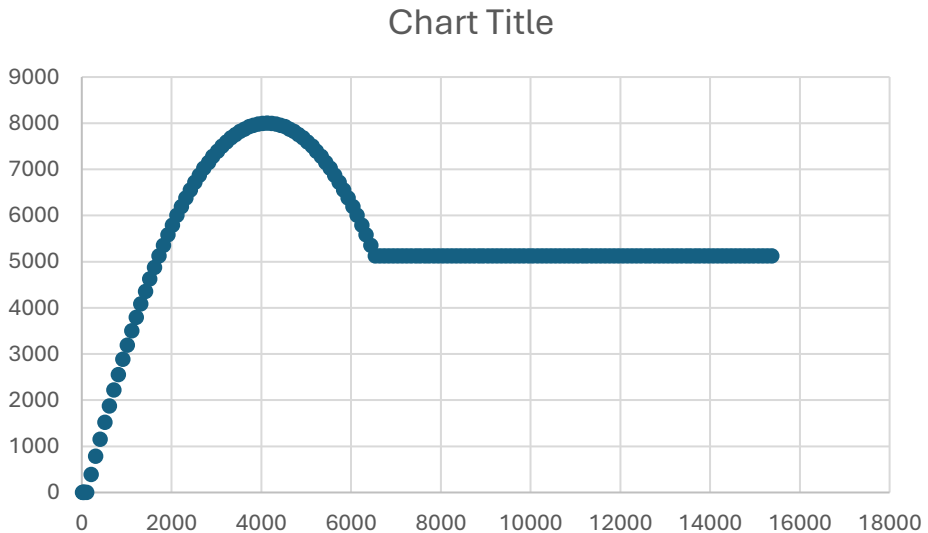
Risultato: l'operatore apre il browser su `http://<ip-raspberry>:8000` e vede direttamente il sito. Niente porte separate, niente `npm run dev`, niente secondo servizio. Un solo `cai-csmt.service` serve tutto.

CONNESSIONE SITO(RASPBERRY) E ARDUINO(CELLA CARICO)
PROVA CHE TUTTO FUNZIONA

- 1. Apro Anaconda Prompt
- 2. Vado nella cartella > cd C:\Users\aleess\Desktop\Cai-CSMT-Simulatore\SITO V2.2\backend
- 3. Attivo ambiente virtuale dove ho i requisiti delle librerie installate, > venv\Scripts\activate
- 4. Avvio l'applicazione ed il server, > python main.py

- 1. Apro un altro terminale Anaconda Prompt
- 2. Vado nella cartella > cd C:\Users\aleess\Desktop\Cai-CSMT-Simulatore\SITO V2.2\frontend
- 3. Faccio partire il sito, >npm run dev
- 4. Su Chrome vado al sito, <http://localhost:5173/>
- 5. Compilo il form e clicco conferma
- 6. Aspetto un tot di secondi e clicco ho fatto la caduta
- 7. Clicco scarica su excel i file csv

	A	B	C	D	E	F	G	H	I
1	timestamp	corda	assicuratore	operatore	picco_N	offset_ms	durata_ms	letture_totali	
2	2025-12-1	(camp	grigri	Pippo	118.97	3973	5689	2263	
3	2026-05-0	10	camp	io	800	4143	15488	159	
4	2026-05-0	100	camp2	sempre io	800	4143	14480	149	
5	2026-05-0	5	Gri-Gri	pippo	800	4129	15372	157	
6									
7									



TO DO: CONNESSIONE CON MVD2555 INVECE CHE ARDUINO

In `sensor_monitor.py`

- 1. Cancellare lo SCENARIO** e tutto il `serial.write(cmd)` di start. Probabilmente all'apertura della seriale l'MVD2555 sta già mandando dati o si interroga con un comando tipo `MSV?1,0` per "mandami i valori in continuo".
- 2. Riscrivere il parsing** delle righe ricevute, in base al formato dell'MVD2555 da manuale.
- 3. Calibrare l'unità di misura.** L'MVD2555 manda mV o V, vanno convertiti in Newton con il coefficiente della cella.
- 4. Eventualmente fare la tara** all'avvio (comando `TAR` dell'MVD2555 per azzerare la baseline).
- 5. Cancellare il `serial.write(b"R")`** allo stop, sostituendolo con quello che serve davvero (probabilmente niente).